MOBILE APPLICATION DEVELOPMENT

Android Location Services — Group Activity Sheet

5 Groups · 5 Mini Apps · Build in ~1 Hour Each

Activity Overview

Your class will split into 5 groups. Each group builds ONE small Android app using the Fused Location Provider concepts covered in class (permissions, current location, distance calculations, continuous updates, and geofencing). Every group presents a 3–5 minute demo at the end of the session.

Ground Rules

- Time limit: 60 minutes of build time, plus 10 minutes for demo prep.
- Every group member must be able to explain the permission flow used.
- Use Kotlin + Fused Location Provider (Google Play services).
- Test on an emulator with simulated location, or a physical device outdoors.
- If you get stuck, log the blocker in the 'Notes / Blockers' section — don't stay silent.

Group Assignment

Group	App Idea	Core Concept
1	"Where Am I?" Locator	One-time location fetch + display
2	Campus / Home Zone Checker	Distance math (distanceTo)
3	Simple Distance Tracker	Two-point location capture
4	Speed Snapshot	Continuous location updates
5	Geofence Alert (Mini)	Geofencing API

GROUP 1

"Where Am I?" Locator

Time budget: 60 minutes build + 10 minutes demo prep

Goal

Fetch the device's current location once and display it clearly on screen.

Key Concepts Applied

Runtime permissions, Fused Location Provider, one-time location request, reading a Location object.

Requirements (must have)

- ☐ Request ACCESS_FINE_LOCATION (or COARSE) permission at runtime.
- ☐ Add a button labeled "Get My Location".
- ☐ On tap, fetch current location using the Fused Location Provider.
- ☐ Display latitude, longitude, accuracy, and timestamp in TextViews.
- ☐ Handle the case where permission is denied (show a message, don't crash).

Build Steps

1. Add location permission to AndroidManifest.xml and the Play services location dependency.
2. Build a simple layout: one button + four text labels (lat, lng, accuracy, time).
3. Check/request permission when the button is tapped.
4. Call getCurrentLocation() on the fused client and update the TextViews in the callback.
5. Test on emulator: set a mock location via Extended Controls, tap the button, verify values update.

Stretch Goals (if time remains)

- Reverse-geocode the coordinates into a readable address using Geocoder.
- Show a small map pin or static map image at that location.

Deliverable / Demo Criteria

A working APK/emulator run that displays real coordinates after tapping the button, plus a 3-minute demo explaining how permission was requested and where the location value came from.

GROUP 2

Campus / Home Zone Checker

Time budget: 60 minutes build + 10 minutes demo prep

Goal

Tell the user whether they are currently inside or outside a fixed zone (e.g., campus, home, classroom) without using the full Geofencing API.

Key Concepts Applied

Fused Location Provider, Location.distanceTo(), simple radius comparison.

Requirements (must have)

- ☐ Hardcode one reference coordinate (latitude/longitude) and a radius in meters.
- ☐ Request location permission and fetch current location on button tap.
- ☐ Calculate the distance between current location and the reference point.
- ☐ Display "Inside Zone" or "Outside Zone" based on whether distance ≤ radius.
- ☐ Show the calculated distance in meters on screen.

Build Steps

6. Pick a reference point (e.g., your campus building) and a sensible radius (e.g., 200m).
7. Build a layout: button + status text + distance text.
8. Request permission and fetch current location on tap.
9. Use location.distanceTo(referenceLocation) to compute distance.
10. Compare distance to radius and update the status text accordingly.
11. Test with the emulator's mock location set both inside and outside the radius.

Stretch Goals (if time remains)

- Let the user tap a spot on a map to set the reference point instead of hardcoding it.
- Change the screen background color depending on inside/outside status.

Deliverable / Demo Criteria

A demo showing the status text flip from "Outside Zone" to "Inside Zone" when the mock location is moved closer than the radius, with the distance value shown both times.

GROUP 3

Simple Distance Tracker

Time budget: 60 minutes build + 10 minutes demo prep

Goal

Capture two location points (a start and a later point) and calculate the distance walked between them.

Key Concepts Applied

Fused Location Provider, storing a Location object across taps, distanceTo().

Requirements (must have)

- ☐ Button 1: "Set Start Point" — captures and stores the current location.
- ☐ Button 2: "Set End Point" — captures current location and computes distance from the stored start point.
- ☐ Display the start coordinates, end coordinates, and total distance (in meters).
- ☐ Disable or handle the End button gracefully if no start point has been set yet.

Build Steps

12. Build a layout: two buttons + text fields for start location, end location, and distance.
13. Request location permission on first use.
14. On "Set Start Point", fetch location and store it in a variable.
15. On "Set End Point", fetch location, then call distanceTo() against the stored start location.
16. Display all values; guard against End being tapped before Start.
17. Test using two different mock locations set a few minutes apart in the emulator.

Stretch Goals (if time remains)

- Convert and display distance in kilometers when it exceeds 1000m.
- Add a "Reset" button to clear both points and start over.

Deliverable / Demo Criteria

A demo where the group sets a start point, changes the mock location, sets an end point, and shows a correct non-zero distance result.

GROUP 4

Speed Snapshot

Time budget: 60 minutes build + 10 minutes demo prep

Goal

Continuously track location updates and display the device's current speed and accuracy live.

Key Concepts Applied

LocationRequest, continuous location updates (callback-based), Location.getSpeed().

Requirements (must have)

- ☐ Request location permission at runtime.
- ☐ Start continuous location updates using a LocationRequest (not a single fetch).
- ☐ Update a TextView live each time a new location arrives, showing speed and accuracy.
- ☐ Provide a "Start" and "Stop" button to begin/end updates (avoid draining battery indefinitely).
- ☐ Stop updates in onPause()/onStop() to follow good lifecycle practice.

Build Steps

18. Build a layout: Start button, Stop button, and a large text label for speed.
19. Request permission, then build a LocationRequest with a short update interval for testing.
20. Register a location callback that updates the speed/accuracy TextView on each result.
21. Wire Start/Stop buttons to begin/remove location updates.
22. Test in the emulator using the route/movement simulation in Extended Controls to generate changing speed values.

Stretch Goals (if time remains)

- Convert speed from m/s to km/h for a more readable display.
- Show a simple color change (e.g., green/yellow/red) based on speed thresholds.

Deliverable / Demo Criteria

A live demo where speed updates on screen as the simulated route plays, and updates visibly stop when Stop is tapped.

GROUP 5

Geofence Alert (Mini)

Time budget: 60 minutes build + 10 minutes demo prep

Goal

Trigger an on-screen alert (toast or notification) when the device enters or exits one fixed geofence.

Key Concepts Applied

Geofencing API, GeofencingClient, GeofencingRequest, geofence transitions (ENTER/EXIT), PendingIntent/BroadcastReceiver.

Requirements (must have)

- ☐ Request location permission (and background location if targeting newer Android versions, time permitting).
- ☐ Define one Geofence with a fixed latitude/longitude and radius.
- ☐ Register the geofence using GeofencingClient with a PendingIntent target.
- ☐ Handle ENTER and EXIT transitions and show a Toast or notification for each.
- ☐ Clearly label on screen which geofence (location + radius) is active.

Build Steps

23. Add geofencing dependency (already included with Play services location) and permissions.
24. Build a layout: a label showing the active geofence and a log/status area.
25. Create the Geofence object and GeofencingRequest (set INITIAL_TRIGGER as needed).
26. Create a BroadcastReceiver (or IntentService) to handle the geofence transition intent and show a Toast/notification.
27. Register the geofence with geofencingClient.addGeofences().
28. Test using the emulator's mock location: move the point across the geofence boundary and confirm the alert fires both ways.

Stretch Goals (if time remains)

- Log a simple in-app history list of enter/exit events with timestamps.
- Support a second geofence and identify which one triggered.

Deliverable / Demo Criteria

A demo showing an alert fire when the mock location crosses into the geofence, and another when it exits.